



ASSIGNMENT NO# 1

Processor State, Arithmetic, Loops and Bitwise Control

Course	Computer Organization and Assembly Language	Assignment	No. 1
Semester	Fall 2026	Total Marks	50
Issue Date	_____	Due Date	_____

Instructions

- *This is an individual assignment. Use the Visual Studio MASM Win32/x86 setup used in the course.*
- *All questions must be solved using only the concepts and instruction forms covered in the Week 3 lectures.*
- *Loops must be constructed using CMP, conditional jumps and JMP. Do not use LOOP, .IF, .ELSE, .WHILE, .REPEAT, macros, or other higher-level control-flow constructs.*
- *For every programming question, submit both the MASM code and a complete handwritten dry run. A final output without the required processor-state trace will receive limited credit.*
- *For every conditional jump, write TAKEN or NOT TAKEN and identify the relevant flag condition. For MUL/IMUL/DIV/IDIV, show the implicit registers and distinguish defined, preserved and undefined flags.*
- *Your code, handwritten trace, verification values and debugger execution must agree. Any part may be checked in a short viva.*

Roll-number Personalization

For a roll number in the form 25i-ABCD, use the four digits after the hyphen as A, B, C and D.

Example: 25i-0123 -> A = 0, B = 1, C = 2, D = 3

Handwritten Trace

For each executed instruction, show the relevant register state, the flags defined by that instruction, and the branch decision where applicable. For loop questions, clearly separate iterations.

Iter.	Instruction	AX	BX	CX	DX	Other reg.	FLAGS	Jump / reason



Question 1: Wide Arithmetic Round Trip

[8 marks]

Use your roll-number digits to calculate:

$$X = 4000h + (A \times 0100h) + (B \times 0010h) + C$$

$$M = 3 + (D \bmod 3)$$

$$DELTA = 10h + C + D$$

Write and dry-run a MASM sequence that performs the following operations in this order:

1. Load AX = X and BX = M.
2. Execute word MUL with BX. Preserve the low 16 bits of the full product in SI and the high 16 bits in DI.
3. Without rebuilding the dividend, execute word DIV with BX using the current DX:AX produced by MUL.
4. Subtract DELTA from the quotient in AX.

Required work

1. Show the complete product after MUL as DX:AX and also as one hexadecimal value.
2. Give CF and OF after MUL; mark PF, AF, ZF and SF appropriately.
3. Show SI and DI after the product is preserved and state whether the MOV instructions change the MUL flag state.
4. Show the DIV quotient and remainder and verify dividend = quotient x divisor + remainder.
5. State the arithmetic-flag status immediately after DIV.
6. Give final AX after SUB and calculate CF, PF, AF, ZF, SF and OF for the final SUB.

Verification Signature	
Full product (DX:AX)	_____
DI : SI	_____
DIV quotient / remainder	_____
Final AX	_____
Final SUB flags	_____



Question 2: Compound-Condition Arithmetic Loop

[10 marks]

Calculate: $START = 10 + A$

$STEP = 3 + (B \bmod 4)$

$LIMIT = 25 + (2 \times C) + D$

$PIVOT = 20 + A + B + C$

$N = 4 + (D \bmod 3)$

Initialize $AX = START$ and $SI = 0$. Construct a loop that continues only while both conditions are true:

$SI < N$ AND $AX < LIMIT$ (unsigned)

Each successful iteration performs the following logic:

```
AX = AX + STEP
if (AX > PIVOT)      ; unsigned
    AX = AX - 2
else
    AX = AX + 1
SI = SI + 1
```

Required work

1. Write the complete MASM loop and IF/ELSE logic.
2. Dry-run every executed instruction for your own values. Clearly mark each loop iteration.
3. For both loop conditions and for the PIVOT decision, identify the comparison being made, the relevant flags, and whether the jump is taken.
4. State how many times the $AX > PIVOT$ branch is true and how many times the ELSE branch executes.
5. Give final AX, final SI, total iterations, and the exact exit reason: LIMIT, N, or both.

Verification Signature	
Iterations	_____
Final AX	_____
Final SI	_____
AX>PIVOT count	_____
ELSE count	_____
Exit reason	_____



Question 3: Bitwise and Shift Processing Loop [10 marks]

Calculate:

$$R = (10 \times A + B) \bmod 16$$

$$\text{STATUS} = A0h + R$$

$$\text{SHIFT} = 1 + (C \bmod 3)$$

$$\text{MASK} = 2^{(D \bmod 4)} \quad ; \quad 01h, 02h, 04h \text{ or } 08h$$

$$N = 3 + (B \bmod 3)$$

$$T = 2 + (A \bmod 4)$$

Initialize AL = STATUS and SI = 0. The loop continues only while:

$$SI < N \quad \text{AND} \quad \text{bit 5 of AL} = 1$$

Inside every iteration, perform the following in order:

1. Use TEST to inspect the bit selected by MASK without changing AL. If the selected bit is 1, toggle bit 6 of AL using XOR; otherwise set bit 0 using OR.
2. Copy AL to AH. Load CL = SHIFT. Shift AH right by CL and mask AH with 07h. This produces a 3-bit field in AH.
3. Evaluate the compound condition: $SI \geq 1 \text{ AND } AH \geq T \text{ AND bit 0 of AL} = 1$. If all three conditions are true, clear bit 5 of AL. Otherwise toggle bit 2 of AL.
4. Increment SI using ADD SI,1 and return to the loop condition.

When the loop terminates, complement AL using NOT, then retain only the low four bits using AND.

Required work

1. Write the complete MASM code using TEST, AND, OR, XOR, NOT, SHR, CMP and conditional jumps as required.
2. Use the Week 3 shift-count form: for a multi-bit count, load the count in CL and use SHR AH,CL.
3. For each iteration, show AL before and after each bitwise operation, the TEST result/ZF, AH after SHR and after AND 07h, and all branch decisions.
4. Show the short-circuit evaluation of all three conditions in the inner compound AND.
5. Record CF after SHR. At termination, state why NOT does not refresh the arithmetic flags, then give the final flags produced by the last AND.
6. Give the exact reason the loop ends: N reached, bit 5 cleared, or both.



Verification Signature	
SHIFT / MASK / T	
Iterations	
AL before NOT	
Final AL	
Final ZF / CF	
Exit reason	



Question 4: Signed Arithmetic Loop with IMUL, IDIV and SAR [12 marks]

Calculate:

$$S = -(20 + 2A + C)$$

$$M = 2 + (B \bmod 2)$$

$$K = 1 + (C \bmod 3)$$

$$V = 3 + (D \bmod 2)$$

$$N = 2 + (A \bmod 2)$$

$$\text{STATUS} = 10h + D$$

Initialize AL = S, DL = STATUS and SI = 0. Repeat the following process exactly N times using CMP and a conditional jump:

Stage A - signed arithmetic

1. Load BL = M and execute byte IMUL BL. Keep the full AX result for the trace.
2. Add K to AL. The next operation uses the actual 8-bit value now present in AL, even if the earlier full IMUL result did not fit in AL.
3. Use CBW to prepare the signed byte value in AL for byte IDIV.
4. Load BL = V and execute IDIV BL. AL becomes the signed quotient and AH the signed remainder.

Stage B - compound decision

Immediately after IDIV, do not rely on the flags left by IDIV. Explicitly generate the flag state needed to implement:

```
if (AL < 0 AND AH != 0)      toggle bit 0 of DL
else if (AL == 0 OR AH == 0) set bit 1 of DL
else                        clear bit 1 of DL
```

Use signed comparison where signed meaning is required and use short-circuit evaluation for both compound conditions.

Stage C - loop and final signed shift

Increment SI using ADD SI,1, compare it with N, and repeat until SI == N. After the final iteration, execute SAR AL,1 and record the final AL and CF.

Required work

1. Show the signed value entering each iteration and the complete AX result after every IMUL.
2. Give CF/OF after IMUL and mark the status of the other arithmetic flags.
3. After ADD, calculate all six arithmetic status flags. Show AX after CBW.
4. After IDIV, give quotient and remainder in signed decimal and hexadecimal and verify dividend = quotient x divisor + remainder.



5. Show the comparisons used after IDIV, the branch path through the IF / ELSE-IF / ELSE chain, and DL before/after its bit operation.
6. Show SI and the loop-back decision for every iteration.
7. For the final SAR AL,1, give the result and CF and explain why SAR is not generally identical to signed IDIV by 2 for negative odd values.

Verification Signature	
Iterations	_____
Final quotient / remainder	_____
Final DL	_____
AL before SAR	_____
AL after SAR	_____
SAR CF	_____



Question 5: Integrated Code Review, Repair and Trace [10 marks]

The following code is intended to satisfy the requirements below, but it contains multiple instruction-form and processor-state errors. You must identify and correct all of them before doing the dry run.

Calculate:

VALUE = (A x 1000h) + (B x 0100h) + (C x 0010h) + D

MULT = 2 + (D mod 3)

LIMIT = 9000h + (C x 0100h) + (D x 0010h)

NEG = -(30 + A + B)

DIVISOR = 3 + (C mod 2)

STATUS = 20h + D

Intended behavior

1. Multiply VALUE by MULT as an unsigned word multiplication.
2. Classify the full product as: zero; too large; or within limit. A nonzero DX means the full product does not fit in AX and is therefore too large. Otherwise compare AX with LIMIT as an unsigned value. All three classification paths must be implemented even if your own roll-number values execute only one of them.
3. Inspect bit 5 of DL without changing DL. If it is set, toggle bit 0.
4. Shift DL left by 3 using the shift-count form required in the Week 3 course subset.
5. Finally calculate the signed word division NEG / DIVISOR using the correct dividend preparation.

Faulty program

Required work

1. Identify all nine problems. For each one, state whether it is an illegal instruction form, wrong flag reasoning, wrong signed/unsigned jump choice, destructive bit test, incorrect shift-count form, missing full-result check, register-state clobbering by an implicit result, or incorrect signed-division preparation.
2. Write the corrected MASM program using only Week 3 instruction forms.
3. Dry-run the corrected program for your roll-number values.
4. Show the full MUL result, the product classification path, DL before/after the bit test/toggle and after SHL, and the final IDIV quotient/remainder.
5. State which instructions create the flags used by every conditional jump.



TA Verification Signature	
Product classification	_____
DX:AX after MUL	_____
DL after SHL	_____
IDIV quotient	_____
IDIV remainder	_____
Nine corrections found	_____